The Cipher Project

MPLADS Transparency & Anomaly Detection System

Complete Technical Architecture & Demo Guide

1. Project Overview & The Problem We Solve

The Member of Parliament Local Area Development Scheme (MPLADS) involves massive data regarding works recommended, completed, and funds expended. However, this data is often siloed, inconsistent, and lacks automated checks for irregularities. **The Cipher Project** bridges this gap by creating an automated analytical pipeline that ingests raw government data, matches records intelligently, and flags anomalies or risk indicators.

> **Key Philosophy (Tell the Judge):** Our system identifies anomalies and risk indicators. It does NOT automatically prove fraud. It points auditors and reviewers exactly where they need to look by providing a mathematically backed **Risk Score (0-100)**.

2. How the System Works (The Big Picture)

Raw Data → Data Cleaning → AI/Fuzzy Matching → Feature Engineering → Anomaly Detection (AI) → Interactive Dashboard

Our system is decoupled into a robust analytical **Backend (Python)** and a responsive **Frontend (React)**.

Backend Architecture

A multi-stage data pipeline that ingests CSVs, cleans text, matches related works, calculates financial features, and uses machine learning (Isolation Forests) to detect statistical anomalies.

Frontend Architecture

A React+Vite dashboard that consumes the backend's exported anomaly data to provide beautiful, actionable insights, charts, and detailed project explorations.

3. The Tech Stack

Backend / Data Science Pipeline:

Python 3 · Pandas (Data manipulation) · Scikit-Learn (Isolation Forest ML) · RapidFuzz (Fuzzy string matching) · Groq (LLM Capabilities)

Frontend / User Interface:

React 19 · Vite (Build Tool) · Tailwind CSS (Styling) · Recharts (Data Visualization) · Lucide React (Icons)

4. Backend Deep Dive: Step-by-Step Pipeline

If the judge asks "How do you process the data?", explain these sequential steps:

Step	What happens under the hood?
1. Cleaning & Normalization	Standardizes dates, amounts, and text. Extracts meaningful entities (like MP names, States) from messy raw data.
2. Multi-Signal Matching	Uses RapidFuzz to match "Recommended Works" with "Completed Works". It checks project descriptions and doesn't rely solely on Work IDs, avoiding errors when IDs are missing or wrong. Matches are categorized into Tier 1 (High Confidence) and Tier 2 (Provisional).
3. Feature Engineering	Calculates timeline deviations, cost overrides, and financial differences relative to peer projects.
4. Anomaly Detection (AI)	Uses **Isolation Forest** (an unsupervised Machine Learning algorithm from Scikit-Learn) to find outliers. It looks for patterns that don't fit the norm (e.g., a project that took way too long or cost disproportionately high).
5. Risk Scoring	Combines ML outputs and financial evidence into a single explainable 0-100 Risk Score. Outputs are saved as JSON/CSV for the frontend.

5. Frontend Deep Dive: What the User Sees

The frontend provides three main views to investigate the data:

- **Dashboard:** A high-level overview with charts (using Recharts). It shows the total number of anomalies, risk distributions, and aggregated state-wise metrics.
- **Projects Explorer:** A searchable, filterable table where users can look up specific works, filter by risk level, and see why a project was flagged.
- **Project Detail:** Clicking a specific project reveals the full "evidence" behind its risk score (e.g., "Cost is 30% higher than peer average").

6. How to Demo to the Judges (Winning Strategy)

> **Step 1: Start with the "Why"**
>
> "Judges, tracking MPLADS funds across thousands of projects manually is impossible. Fraud and delays slip through the cracks. The Cipher Project solves this by acting as an AI auditor."
>
> **Step 2: Show the Frontend Dashboard**
>
> "This is what a government auditor sees. Instead of reading spreadsheets, they see that out of 10,000 projects, 50 require immediate attention."
>
> **Step 3: Dive into a High-Risk Project**
>
> "Let's look at this specific project with a 95/100 risk score. As you can see on the Project Details page, the AI didn't just flag it; it provided Explainable AI reasons: The vendor concentration was unusual, and the cost deviated 40% from similar projects."
>
> **Step 4: Explain the Tech Magic (Backend)**

> "How did we know this? We didn't just do basic SQL. We built a Python pipeline that uses fuzzy string matching (RapidFuzz) to link messy datasets, and an Unsupervised Machine Learning model (Isolation Forest) to detect multi-dimensional anomalies that human eyes would miss."

7. Key Questions Judges Might Ask (And How to Answer)

Q: Is it just rule-based checking?
A: No, while we have rule-based financial checks, we also use Isolation Forest (ML) to catch complex, non-obvious anomalies based on peer comparisons.

Q: What if the raw data is incorrect?
A: We don't overwrite raw data. Our pipeline treats IDs as corroborating evidence, not absolute truth. We use fuzzy text matching on project descriptions to handle data entry typos.